

Customer y Subscription

Guia de implementacion

DDD, EDA, CQRS, SOLID y Clean Architecture
alineados al estado real de TaxVision

Incluye el modelo final de Customer con relaciones y perfiles
fiscales separados, y el onboarding Subscription -> Tenant.

Contenido

1. Alcance y decisiones definitivas	4
1.1 Fuentes revisadas	4
2. Convenciones obligatorias de TaxVision	4
2.1 Definicion de terminado por microservicio	5
PARTE I - Customer Service	6
3. Limite del bounded context Customer	6
3.1 Por que CustomerNotes no pertenece aqui	6
4. Modelo de dominio final de Customer	6
4.1 Aggregate root	6
4.2 CustomerRelation: dos ejes, no un enum ambiguo	6
4.3 Perfiles fiscales separados	7
4.4 Tablas finales del write model	8
5. Invariantes de Customer	8
5.1 Esqueleto de factory	8
6. Implementacion de Customer paso a paso	9
Paso 1 - Crear los cuatro proyectos	9
Paso 2 - Crear la estructura de dominio	9
Paso 3 - Contratos Application	9
Paso 4 - DbContext y filtro tenant	9
Paso 5 - Configuracion EF e indices	10
Paso 6 - Primera migracion	10
Paso 7 - API y autorizacion	10
Paso 8 - Integracion Customer con Auth	10
Paso 9 - Eventos V1	11
Paso 10 - Wolverine, health y observabilidad	11
7. Migracion legacy de Customer	11
7.1 Orden ETL	11
7.2 Pruebas obligatorias de Customer	12
PARTE II - Subscription Service	13
8. Limite del bounded context Subscription	13
9. Onboarding: la suscripcion crea el tenant comercial	13
9.1 Idempotencia del flujo	13

10. Modelo de dominio final de Subscription	14
10.1 Aggregates	14
10.2 Estados	14
10.3 Tablas recomendadas	14
10.4 Money y PricingSnapshot	15
11. Implementacion de Subscription paso a paso	15
Paso 1 - Scaffolding uniforme	15
Paso 2 - Estructura	15
Paso 3 - Factories e invariantes	15
Paso 4 - Commands y queries	16
Paso 5 - Proration	16
Paso 6 - Scheduled messages, no Quartz	16
Paso 7 - Eventos	16
Paso 8 - Entitlements y limite de usuarios	17
Paso 9 - API	17
Paso 10 - Persistencia y migracion	17
12. Migracion legacy de Subscription	17
12.1 Estrategia de datos	18
12.2 Pruebas obligatorias de Subscription	18
13. Integracion completa entre servicios	19
13.1 Reglas de seguridad cross-service	19
14. Orden de implementacion recomendado	19
14.1 Checklist de PR	20
15. Decisiones que no deben revertirse	20
16. Resumen ejecutivo de implementacion	20

1. Alcance y decisiones definitivas

Esta guía describe como implementar Customer y Subscription sobre el estado real del repositorio TaxVision al 27-06-2026. No replica mecanicamente el legacy: conserva sus casos de negocio y corrige sus limites de bounded context, seguridad, tenancy y mensajería.

Resultado esperado. Ambos servicios usan cuatro proyectos, base de datos propia, Wolverine para CQRS y outbox/inbox, RabbitMQ, JWT firmado, CorrelationId, health checks, Redis solo donde aporta valor, y OpenTelemetry. No comparten bases ni realizan joins remotos.

Decision	Aplicacion final
Customer central	Customer es aggregate root y posee toda la informacion intrinseca del cliente, sus direcciones, canales y relaciones.
Fiscal separado	CustomerFiscalProfile es una entidad 1:0..1. Una relacion fiscalmente relevante puede tener CustomerRelationFiscalProfile.
Relacion unificada	CustomerRelation sustituye Spouse, Dependent y CustomerContact. Separa parentesco de proposito operacional.
Notas fuera	Notas, timeline, tareas y actividad operacional pertenecen a un futuro CustomerOperations o Engagement Service.
Portal fuera	Customer no guarda PortalUserId. Auth guarda User.CustomerId y emite customer_id en el JWT.
Tenant reservado	TaxVision Platform es Kind=Platform y queda excluido de Customer, Subscription y Billing.
Subscription antes de tenant	Un SubscriptionEnrollment pre-tenant coordina pago o trial; solo despues solicita el tenant comercial.

Regla de lectura: cuando esta guía dice que Customer lo contiene todo, significa todo lo que forma su identidad y expediente maestro. No significa absorber autenticacion, facturacion, notas operativas, marketing o archivos.

1.1 Fuentes revisadas

- Repositorio D:\TaxVision completo, incluido README, configuracion, migraciones y cambios de timezone e invitaciones.
- CRMTAXPROBACKEND.zip: entidades Customer, Auth Invitations y Subscription legacy.
- Propuesta_Customer_Microservicio_TaxVision.pdf, 34 paginas.
- Guia_Migracion_CRMTaxPro_a_TaxVision.pdf y Guia_TaxVision_Implementacion_Completa.pdf.
- Decisiones posteriores de la conversacion: CustomerNotes fuera, PortalUserId fuera, Profile futuro, fiscal separado y relaciones unificadas.

2. Convenciones obligatorias de TaxVision

Area	Convencion
Estructura	Service.Domain, Service.Application, Service.Infrastructure y Service.Api.
Dominio	Factories y metodos con setters privados; Result para fallos esperados.
CQRS	Commands y queries despachados por IMessageBus de Wolverine.
Persistencia	SQL Server, DbContext por servicio, IRepository estrecho e IUnitOfWork.
Tenancy	TenantId tomado del contexto confiable; nunca del header libre del cliente.
EDA	Eventos V1, outbox en productor, inbox durable e idempotencia de negocio en consumidor.
Seguridad	JWT con sub, tenant_id, actor_type, zoneinfo, roles y customer_id cuando corresponde.
Tiempo	Instantes en UTC; identificadores IANA para conversion de zona horaria.
Observabilidad	CorrelationId, Serilog, OTLP, health live/ready y metricas de lag.
Despliegue	Dockerfile con versiones exactas, ruta YARP y servicio en taxvision-network.

```
API -> Application -> Domain
|           ^
v           |
Infrastructure

Write -> SQL + Wolverine outbox -> RabbitMQ
Consumer -> Wolverine inbox -> read model local
```

2.1 Definicion de terminado por microservicio

- La solucion compila sin warnings.
- La migracion inicial y el script idempotente de EF se generan correctamente.
- Toda tabla tenant-owned tiene TenantId e indices que comienzan por TenantId en los hot paths.
- Los endpoints anonimos estan explicitamente marcados y limitados por rate limiting.
- Los eventos incluyen EventId, TenantId, OccurredOn y CorrelationId.
- Los consumidores son reentrantes y verifican el estado de negocio antes de mutar.
- README, archivo .http, Postman y matriz de autorizacion reflejan el codigo real.
- Existen pruebas unitarias de invariantes, integracion con SQL/Rabbit y E2E del slice critico.

PARTE I - Customer Service

3. Limite del bounded context Customer

Customer es el sistema de registro del cliente fiscal dentro de una oficina. Administra identidad de persona o negocio, contacto maestro, direcciones, relaciones y perfil fiscal. Cada dato tiene TenantId y nunca puede cruzar oficinas.

Permanece en Customer	Salen de Customer
Identidad individual o de negocio	Credenciales y sesiones -> Auth
Email, telefono y canales adicionales	PortalUserId -> no se almacena en Customer
Direcciones	Notas, tareas, timeline -> CustomerOperations/Engagement
Relaciones: conyuge, dependiente, contacto, socio	Consentimiento marketing -> Campaign
CustomerFiscalProfile	Facturas y pagos -> Billing/Payment
CustomerRelationFiscalProfile	Vehiculos y viajes -> Tracking
Estado y preferencias maestras	Bytes y rutas de archivo -> CloudStorage

3.1 Por que CustomerNotes no pertenece aqui

Una nota describe trabajo realizado sobre un cliente, no la identidad del cliente. Su ciclo de vida depende de casos, tareas, comunicaciones, adjuntos y auditoria operacional. Incluirla en el aggregate Customer obligaria a cargar y bloquear el expediente maestro cada vez que alguien agrega una observacion.

- Customer publica CustomerCreatedV1 y CustomerUpdatedV1.
- CustomerOperations mantiene un read model minimo del cliente.
- La nota guarda CustomerId, TenantId, CreatedByUserId, cuerpo, categoria y AttachmentFileId.
- Customer no mantiene una coleccion CustomerNotes ni recibe PortalUserId.

4. Modelo de dominio final de Customer

4.1 Aggregate root

```

Customer : TenantEntity
|- CustomerKind           Individual | Business
|- CustomerStatus         Active | Inactive | Archived
|- DisplayName / LegalName
|- PersonalName?          solo Individual
|- BusinessIdentity?      solo Business
|- DateOfBirth?
|- PreferredChannel
|- Language
|- PrimaryEmail / PrimaryPhone
|- ProfilePictureFileId?
|- CustomerFiscalProfile? entidad 1:0..1
|- Addresses[]            child entities
|- ContactPoints[]        child entities
~- Relations[]            CustomerRelation child entities

```

El aggregate expone colecciones de solo lectura. Las altas, cambios y bajas pasan por metodos como AddAddress, AddContactPoint, AddRelation y SetFiscalProfile. EF Core usa backing fields, siguiendo el mismo principio de setters privados de Tenant y User.

4.2 CustomerRelation: dos ejes, no un enum ambiguo

La propuesta de una sola entidad de relacion es correcta, con una correccion importante: parentesco y uso operacional no son la misma cosa. Esposa o hijo describen RelationshipKind; contacto de emergencia o representante describen RelationPurpose. Separarlos evita valores absurdos como un enum que mezcle Spouse con PrimaryContact.

Campo	Tipo	Ejemplos / regla
Id, TenantId, CustomerId	Guid	Identidad y aislamiento.
RelationshipKind	enum string	Spouse, Child, Parent, Friend, Partner, Employee, Shareholder, Other.
Purposes	flags enum	Dependent, TaxHouseholdMember, EmergencyContact, AuthorizedRepresentative, BusinessContact, BeneficialOwner.
Name	PersonalName VO	First, Middle, Last, Prefix y Suffix.
PrimaryEmail	EmailAddress?	Opcional y validado.
PrimaryPhone	PhoneNumber?	Opcional, E.164.
DateOfBirth	DateOnly?	Solo cuando el caso lo requiere.
Address	Address VO?	Columnas owned dentro de CustomerRelations para no crear otra tabla.
FiscalProfile	1:0..1	Solo conyuge, dependiente u otra relacion fiscalmente relevante.
IsActive	bool	Desactivacion logica sin borrar historico.

```
[Flags]
public enum RelationPurpose
{
    None           = 0,
    Dependent      = 1,
    TaxHouseholdMember = 2,
    EmergencyContact = 4,
    AuthorizedRepresentative = 8,
    BusinessContact = 16,
    BeneficialOwner = 32
}
```

Ejemplo: una esposa puede tener RelationshipKind.Spouse y los propósitos TaxHouseholdMember | EmergencyContact. Un CFO de una empresa puede tener RelationshipKind.Employee y AuthorizedRepresentative | BusinessContact.

4.3 Perfiles fiscales separados

CustomerFiscalProfile no es un value object embebido. Tiene reglas, permisos y frecuencia de cambio propios, contiene PII de alta sensibilidad y puede consultarse con autorización distinta. Por eso se modela como entidad 1:0..1 y tabla separada.

CustomerFiscalProfile	CustomerRelationFiscalProfile
CustomerId PK/FK y TenantId	CustomerRelationId PK/FK y TenantId
FiscalSubjectKind: Individual o Business	TaxRelationshipRole: Spouse, Dependent, Other
SSN/ITIN/EIN cifrado y últimos 4 para búsqueda	SSN/ITIN cifrado y últimos 4
FilingStatus, prior-year AGI, returning flag	Eligibility flags y tax year
Refund banking solo si es imprescindible y cifrado	Sin datos bancarios
UpdatedAtUtc y UpdatedByUserId	UpdatedAtUtc y UpdatedByUserId

No registrar identificadores fiscales en logs, eventos generales, trazas o read models. Los eventos CustomerCreatedV1 usan DisplayName y contacto mínimo; los datos fiscales requieren un contrato separado y consumidores expresamente autorizados.

4.4 Tablas finales del write model

Tabla	Cardinalidad	Responsabilidad
Customers	1	Identidad central, estado y contacto primario.
CustomerFiscalProfiles	0..1 por Customer	Informacion fiscal sensible del sujeto principal.
CustomerAddresses	0..N	Home, Mailing, Business, Previous, Foreign, etc.
CustomerContactPoints	0..N	Emails y telefonos adicionales; uno primario por tipo.
CustomerRelations	0..N	Conyuges, dependientes, contactos, socios y representantes.
CustomerRelationFiscalProfiles	0..1 por relacion	Datos fiscales sensibles de relaciones relevantes.

Occupations y codigos de actividad pueden iniciar como enums o catalogos seed globales. Solo se crea una tabla tenant-owned si la oficina necesita valores personalizados. CustomerImportAttempt es operacional y debe agregarse cuando se implemente import masivo, no en el MVP CRUD.

5. Invariantes de Customer

- CustomerKind.Individual exige PersonalName; Business exige LegalName y no obliga a una persona ficticia.
- Email y telefono se normalizan antes de comparar o persistir.
- No puede existir mas de una direccion primaria del mismo AddressKind critico.
- No puede existir mas de un ContactPoint primario por tipo.
- Una relacion con Purpose.Dependent o TaxHouseholdMember puede tener fiscal profile; un contacto simple no lo necesita.
- El mismo SSN/ITIN cifrado debe detectarse mediante un blind index o hash HMAC por tenant, nunca con SHA-256 sin clave.
- Archive es reversible solo mediante una operacion autorizada y publica CustomerArchivedV1.
- TenantId nunca llega desde el body; se obtiene de ITenantContext.
- Customer no puede pertenecer al tenant reservado Platform.

5.1 Esqueleto de factory

```
public static Result<Customer> Register(
    Guid tenantId,
    CustomerKind kind,
    PersonalName? personalName,
    string? legalName,
    EmailAddress primaryEmail,
    PhoneNumber? primaryPhone,
    Guid createdByUserId)
{
    if (tenantId == PlatformTenant.Id)
        return Failure("Customer.PlatformTenant");

    // Validar combinacion Individual/Business.
    // Normalizar contacto y crear estado Active.
    // No recibir CustomerId, TenantId alternativo ni PortalUserId.
}
```


6. Implementacion de Customer paso a paso

Paso 1 - Crear los cuatro proyectos

```
dotnet new classlib -n TaxVision.Customer.Domain `
-o src/Services/Customer/TaxVision.Customer.Domain
dotnet new classlib -n TaxVision.Customer.Application `
-o src/Services/Customer/TaxVision.Customer.Application
dotnet new classlib -n TaxVision.Customer.Infrastructure `
-o src/Services/Customer/TaxVision.Customer.Infrastructure
dotnet new webapi -n TaxVision.Customer.Api `
-o src/Services/Customer/TaxVision.Customer.Api

dotnet sln TaxVision.slnx add src/Services/Customer/**/**/*.csproj
```

Copiar las versiones exactas de EF Core, Wolverine, Swagger, OpenTelemetry y JwtBearer desde Tenant/Auth. No introducir MediatR, AutoMapper ni un segundo stack de logging.

Paso 2 - Crear la estructura de dominio

```
TaxVision.Customer.Domain/
|- Customers/Custom.cs
|- Customers/CustomKind.cs
|- Customers/CustomStatus.cs
|- Customers/ValueObjects/
|   |- PersonalName.cs
|   |- EmailAddress.cs
|   |- PhoneNumber.cs
|   `-- AddressValue.cs
|- FiscalProfiles/CustomFiscalProfile.cs
|- Relations/CustomRelation.cs
|- Relations/RelationshipKind.cs
|- Relations/RelationPurpose.cs
|- Relations/CustomRelationFiscalProfile.cs
|- ContactPoints/CustomContactPoint.cs
`-- Addresses/CustomAddress.cs
```

Los value objects validan formato y normalizacion. Las entidades tienen constructor privado para EF y factories Result<T>. No agregar navigation properties a Auth, Tenant, Payment o CloudStorage.

Paso 3 - Contratos Application

- ICustomerRepository: GetByIdAsync, AddAsync y búsquedas estrictamente necesarias.
- ICustomerReadService: proyecciones paginadas AsNoTracking; nunca retornar el aggregate completo.
- ISensitiveDataProtector: cifrado autenticado y blind index; implementacion en Infrastructure.
- Commands: CreateCustomer, UpdateCustomer, ArchiveCustomer, AddAddress, AddContactPoint, AddRelation, SetCustomerFiscalProfile, SetRelationFiscalProfile.
- Queries: GetCustomerById y SearchCustomers con pagina, size y term normalizado.

Paso 4 - DbContext y filtro tenant

```
public sealed class CustomerDbContext : DbContext, IUnitOfWork
{
    private readonly ITenantContext _tenant;

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(CustomerDbContext).Assembly);
        modelBuilder.Entity<Customer>()
            .HasQueryFilter(x => x.TenantId == _tenant.TenantId);
        modelBuilder.Entity<CustomerRelation>()
            .HasQueryFilter(x => x.TenantId == _tenant.TenantId);
        // Repetir para cada entidad tenant-owned.
    }
}
```

Para jobs y consumers sin request HTTP, el handler debe establecer un contexto de tenant desde el evento antes de consultar. No desactive filtros globales salvo en un repositorio administrativo interno, explícitamente autorizado y auditado.

Paso 5 - Configuración EF e índices

Tabla	Índices mínimos
Customers	UNIQUE (TenantId, NormalizedPrimaryEmail) si el negocio exige unicidad; (TenantId, Status, DisplayName).
CustomerAddresses	(TenantId, CustomerId, Kind); índice filtrado para primarias.
CustomerContactPoints	(TenantId, CustomerId, Type); unique filtrado para principal por tipo.
CustomerRelations	(TenantId, CustomerId, RelationshipKind); (TenantId, NormalizedEmail).
FiscalProfiles	UNIQUE CustomerId; (TenantId, TaxIdentifierBlindIndex) sin almacenar texto claro.

Enums se persisten como string. Valores monetarios usan decimal con precisión explícita. DateOnly usa date; instantes usan datetime2 UTC. Configurar cascade solo dentro del aggregate y Restrict hacia referencias externas.

Paso 6 - Primera migración

```
dotnet ef migrations add InitialCustomer `
  --project src/Services/Customer/TaxVision.Customer.Infrastructure `
  --startup-project src/Services/Customer/TaxVision.Customer.Api

dotnet ef migrations script --idempotent `
  --project src/Services/Customer/TaxVision.Customer.Infrastructure `
  --startup-project src/Services/Customer/TaxVision.Customer.Api
```

Paso 7 - API y autorización

Endpoint	Actor mínimo	Notas
POST /customers	TenantEmployee	TenantId desde contexto; createdBy desde sub.
GET /customers	TenantEmployee	Paginado, AsNoTracking y DTO mínimo.
GET /customers/{id}	TenantEmployee	Nunca expone PII fiscal sin permiso específico.
PATCH /customers/{id}	TenantEmployee	Métodos del aggregate, no SetValues.
POST /customers/{id}/relations	TenantEmployee	RelationshipKind y Purposes separados.
PUT /customers/{id}/fiscal-profile	TenantAdmin o policy fiscal	Audit obligatorio.
POST /customers/{id}/portal-invitations	TenantAdmin	Publica solicitud o usa el contrato Auth acordado.
POST /customers/{id}/archive	TenantAdmin	Publica CustomerArchivedV1.

Paso 8 - Integración Customer con Auth

La fuente de la identidad de portal es Auth. Customer no guarda UserId. Auth guarda CustomerId en User y lo emite como customer_id. El endpoint de Customer solo solicita una invitación; la aceptación y password viven completamente en Auth.

```
TenantAdmin -> Customer: RequestPortalInvitation(customerId, email)
Customer -> event: CustomerPortalInvitationRequestedV1
Auth -> creates Invitation(CustomerPortal, CustomerId)
User -> Auth: AcceptInvitation(token, name, password)
Auth -> JWT: tenant_id + actor_type=CustomerPortal + customer_id

Customer API:
if actor_type == CustomerPortal:
    requestedCustomerId must equal claim customer_id
```

Mientras Customer aun no exista, Auth permite que TenantAdmin cree la invitacion con CustomerId. Al implementar Customer, preferir el evento anterior o una proyeccion local en Auth para validar que el CustomerId pertenece al mismo tenant.

Paso 9 - Eventos V1

Publica	Consumidores tipicos
CustomerCreatedV1	CustomerOperations, Campaign, Billing, Planner, Audit.
CustomerUpdatedV1	Actualiza read models locales; solo campos no sensibles.
CustomerArchivedV1	Desactiva operaciones futuras conservando historico.
CustomerEmailChangedV1	Email, Campaign y Signature.
CustomerPhoneChangedV1	Campaign y SMS; reevalua consentimientos.
CustomerPortalInvitationRequestedV1	Auth y futuro Email/Notification.
CustomersBulkImportedV1	Lotes acotados; consumidores usan insercion masiva.

Los contratos no incluyen SSN, ITIN, EIN completo, cuentas bancarias ni cuerpos de notas. Para consumidores fiscales use un contrato separado, cifrado en transito, con una lista de consumidores autorizados y revision de seguridad.

Paso 10 - Wolverine, health y observabilidad

- PersistMessagesWithSqlServer y UseDurableOutboxOnAllSendingEndpoints.
- UseEntityFrameworkCoreTransactions con CustomerDbContext e IUnitOfWork.
- Una cola durable propia por consumidor, no una cola compartida por todos.
- Health ready para SQL Server, RabbitMQ y Redis solo si Customer realmente usa cache.
- Metricas: customer_created_total, customer_search_duration, customer_event_lag_seconds y fiscal_profile_access_total.
- Logs con CustomerId permitido; identificadores fiscales y datos bancarios prohibidos.

7. Migracion legacy de Customer

Legacy	Destino
Customers	Customers + CustomerFiscalProfiles segun sensibilidad.
ContactInfos	PrimaryEmail/Phone + CustomerContactPoints; credenciales descartadas/migradas a Auth.
Addresses	CustomerAddresses.
Spouses	CustomerRelations con RelationshipKind.Spouse + fiscal profile opcional.
Dependents	CustomerRelations con Purpose.Dependent + CustomerRelationFiscalProfile.
CustomerContacts	CustomerRelations con BusinessContact, EmergencyContact o AuthorizedRepresentative.
TaxInformations	CustomerFiscalProfiles.
Notes	CustomerOperations/Engagement, no Customer.
Invoices/Products/Payments	Billing/Payment.
Vehicles/Trips	Tracking.
PendingFolderOperations	No migrar; sustituir por CloudStorage y reconciliar pendientes.

7.1 Orden ETL

- Congelar el diccionario legacy y crear una tabla de mapeo LegacyCustomerId -> CustomerId.
- Migrar Customers sin PII fiscal y validar conteos por CompanyId/TenantId.

- Migrar perfiles fiscales cifrando en el proceso; comparar ultimos cuatro y conteos, nunca dumps en texto.
- Migrar direcciones y puntos de contacto.
- Unificar spouse, dependents y contacts en CustomerRelations con reglas deterministas.
- Publicar backfill events por lotes despues de confirmar la base, no dentro de cada INSERT ETL.
- Ejecutar reconciliacion por tenant, muestreo funcional y ventana de cutover con rollback.

7.2 Pruebas obligatorias de Customer

Nivel	Casos
Unitarias	Individual/Business, relacion multiproposito, duplicados primarios, fiscal profile permitido, Platform tenant rechazado.
Integracion	Filtros TenantId, indices, cifrado/blind index, cascades y outbox atomico.
Consumer	Duplicados, eventos fuera de orden, archived y bulk batches.
E2E	Crear Customer, invitar CustomerPortal, aceptar, login y acceso solo al customer_id firmado.
Arquitectura	Domain sin EF/ASP.NET; Application sin Infrastructure; contratos V1 sin PII.

PARTE II - Subscription Service

8. Limite del bounded context Subscription

Subscription decide que plan, periodo, limites y features corresponden a una cuenta SaaS. No cobra tarjetas, no emite facturas y no es propietario del tenant. Coordina esos servicios mediante estados y eventos.

Subscription posee	Otro bounded context posee
Plan, PlanVersion, feature entitlement	Tenant: identidad y ciclo de vida de la oficina.
SubscriptionEnrollment pre-tenant	Payment: intentos, providers, transacciones y refunds.
Subscription activa y periodos	Billing: invoices, taxes, receipts y accounting.
Cambios pendientes, cancel-at-period-end	Auth: usuarios, invitaciones y sesiones.
PricingSnapshot aceptado	Notification: emails y avisos.

El tenant reservado TaxVision Platform nunca recibe una Subscription. Todos los handlers y consumers rechazan o ignoran TenantKind.Platform.

9. Onboarding: la suscripcion crea el tenant comercial

Para cumplir la decision de crear cada tenant despues de la suscripcion, se necesita una entidad pre-tenant. No debe inventarse un TenantId antes de que Tenant Service lo cree. SubscriptionEnrollment usa EnrollmentId y mantiene los datos minimos de onboarding hasta completar pago o trial.

```

1. POST /subscription-enrollments
   -> selecciona PlanVersion, email admin, nombre, subdominio y timezone
2. Subscription crea Enrollment.PendingPayment
   -> publica SubscriptionPaymentRequestedV1
3. Payment confirma -> PaymentCompletedV1
4. Subscription marca Enrollment.PaymentConfirmed
   -> publica TenantProvisioningRequestedV1
5. Tenant crea Kind.Customer + invitacion TenantAdmin
   -> publica TenantCreatedIntegrationEvent
6. Subscription vincula Enrollment.TenantId
   -> crea Subscription.Active o Trialing
   -> publica SubscriptionActivatedV1
7. Auth recibe TenantCreated y crea Invitation(TenantAdmin)
8. Cliente acepta invitacion y entra a su oficina

```

En un trial sin cobro, el paso 2 registra la politica de trial y salta a TenantProvisioningRequestedV1. Si el provisioning falla, Enrollment permanece PaymentConfirmed/PendingProvisioning y Wolverine reintenta sin cobrar otra vez.

9.1 Idempotencia del flujo

- PaymentCompletedV1 incluye PaymentId e EnrollmentId; un indice unico evita procesarlo dos veces.
- TenantProvisioningRequestedV1 lleva EnrollmentId como ProvisioningKey.
- Tenant Service debe devolver el mismo TenantId si recibe otra vez la misma ProvisioningKey.
- TenantCreated se procesa con upsert; Auth y Subscription usan inbox durable.
- SubscriptionActivatedV1 solo se publica en la primera transicion a Active/Trialing.
- No ejecutar compensacion destructiva del tenant por un fallo posterior de email.

10. Modelo de dominio final de Subscription

10.1 Aggregates

Aggregate / entidad	Alcance	Responsabilidad
Plan	Global control plane	Nombre, estado y conjunto de versiones. No TenantId.
PlanVersion	Global immutable	Precio, moneda, periodo, user limit, features y vigencia.
SubscriptionEnrollment	Pre-tenant	Selección, admin, subdominio, timezone, PaymentId y provisioning.
Subscription	Tenant-owned	PlanVersion actual, status, periodos, auto-renew y cancelación.
PendingSubscriptionChange	Child de Subscription	Upgrade inmediato o downgrade al cierre; snapshot y proration.

Transaction, TransactionHistory, PaymentToken y provider data del legacy no entran en Subscription. Viven en Payment. RenewalNotificationLog vive en Notification o Audit. Subscription conserva referencias externas e idempotency keys, no duplica el ledger.

10.2 Estados

```

Enrollment:
Draft -> PendingPayment -> PaymentConfirmed -> PendingProvisioning
      -> Completed
      -> Failed | Cancelled

Subscription:
Trialing -> Active -> PastDue -> Active
          |           |
          v           v
        Suspended  CancelAtPeriodEnd -> Cancelled
  
```

Estado	Permite acceso	Regla
Trialing	Si	Hasta TrialEndsAtUtc; límites del plan.
Active	Si	Periodo vigente y pago confirmado o plan gratuito.
PastDue	Periodo de gracia	No suspender inmediatamente; Payment reintenta.
Suspended	No o acceso reducido	Publica entitlement revocation; no borra datos.
CancelAtPeriodEnd	Si hasta fin	No renovar; downgrade/cancelación diferida.
Cancelled	No	Estado terminal salvo reactivación explícita con nuevo pago.

10.3 Tablas recomendadas

Tabla	Claves e índices
Plans	PlanId; Code unique; IsActive. Global, sin TenantId.
PlanVersions	PlanVersionId; (PlanId, Version) unique; EffectiveFrom; snapshot immutable.
PlanFeatures	(PlanVersionId, FeatureCode) unique; LimitValue opcional.
SubscriptionEnrollments	EnrollmentId; Email; Status; PaymentId unique filtrado; ProvisioningKey unique.
Subscriptions	SubscriptionId; TenantId unique; Status; CurrentPeriodEndUtc.
PendingSubscriptionChanges	ChangeId; SubscriptionId; Status; EffectiveAtUtc; PaymentId/idempotency key.

10.4 Money y PricingSnapshot

Money usa Amount decimal(19,4) y Currency ISO-4217. PlanVersion es immutable. Subscription copia un PricingSnapshot para que una subida futura del catalogo no reescriba el contrato historico del tenant.

```
public sealed record Money(decimal Amount, string Currency);

public sealed record PricingSnapshot(
    Guid PlanVersionId,
    Money RecurringPrice,
    BillingPeriod BillingPeriod,
    int IncludedUsers,
    IReadOnlyCollection<string> FeatureCodes);
```

11. Implementacion de Subscription paso a paso

Paso 1 - Scaffolding uniforme

```
dotnet new classlib -n TaxVision.Subscription.Domain `
-o src/Services/Subscription/TaxVision.Subscription.Domain
dotnet new classlib -n TaxVision.Subscription.Application `
-o src/Services/Subscription/TaxVision.Subscription.Application
dotnet new classlib -n TaxVision.Subscription.Infrastructure `
-o src/Services/Subscription/TaxVision.Subscription.Infrastructure
dotnet new webapi -n TaxVision.Subscription.Api `
-o src/Services/Subscription/TaxVision.Subscription.Api
```

Reusar exactamente los paquetes y extensiones del repositorio. Subscription necesita SQL, RabbitMQ y normalmente Redis para catalogo de planes; no necesita acceso a TaxVision_Tenants ni TaxVision_Auth.

Paso 2 - Estructura

```
TaxVision.Subscription.Domain/
|- Plans/Plan.cs
|- Plans/PlanVersion.cs
|- Plans/PlanFeature.cs
|- Enrollments/SubscriptionEnrollment.cs
|- Subscriptions/Subscription.cs
|- Subscriptions/PendingSubscriptionChange.cs
|- Subscriptions/SubscriptionStatus.cs
|- ValueObjects/Money.cs
|- ValueObjects/PricingSnapshot.cs
|- ValueObjects/BillingPeriod.cs

TaxVision.Subscription.Application/
|- Abstractions/
|- Enrollments/Commands/
|- Plans/Commands,Queries/
|- Subscriptions/Commands,Queries/
|- Payments/IntegrationEvents/
|- Tenants/IntegrationEvents/
```

Paso 3 - Factories e invariantes

- PlanVersion publicada es immutable; cualquier cambio crea una version nueva.
- Enrollment exige plan activo, email valido, subdominio valido y timezone IANA.
- Solo PaymentCompleted puede confirmar un enrollment pagado.
- Subscription exige TenantId comercial y unique; rechaza PlatformTenant.Id.
- Upgrade inmediato requiere PaymentCompleted para la diferencia; downgrade aplica al final del periodo.
- CancelAtPeriodEnd no corta acceso antes de CurrentPeriodEndUtc.
- Renew es idempotente por PaymentId y periodo.

Paso 4 - Commands y queries

Feature	Commands / queries
Planes	CreatePlan, PublishPlanVersion, RetirePlanVersion, ListPublicPlans.
Enrollment	StartEnrollment, CancelEnrollment, GetEnrollmentStatus.
Provisioning	ConfirmEnrollmentPayment consumer, BindProvisionedTenant consumer.
Ciclo de vida	StartTrial, Activate, MarkPastDue, Suspend, CancelAtPeriodEnd, Reactivate.
Cambios	RequestPlanChange, ConfirmPlanChangePayment, ApplyScheduledChange.
Lecturas	GetCurrentSubscription, GetEntitlements, GetRenewalPreview.

Paso 5 - Proration

Subscription calcula el ajuste, pero Payment ejecuta el cobro. La formula usa instantes UTC y la politica comercial definida; no use dias enteros si el negocio factura por segundo.

```
remainingRatio =
    (CurrentPeriodEndUtc - now).TotalSeconds
    / (CurrentPeriodEndUtc - CurrentPeriodStartUtc).TotalSeconds;

credit = currentPrice * remainingRatio;
charge = newPrice * remainingRatio;
amountDue = Max(0, charge - credit);

// Guardar el resultado en PendingSubscriptionChange.
// Publicar PlanChangePaymentRequestedV1.
// Aplicar solo tras PaymentCompletedV1 con la misma ChangeId.
```

Redondear Money una sola vez segun la moneda. Guardar currentPrice, newPrice, ratio, credit, charge, amountDue y policy version para auditoria.

Paso 6 - Scheduled messages, no Quartz

- Programar SubscriptionRenewalDue al crear o renovar el periodo.
- El handler verifica SubscriptionId, CurrentPeriodEndUtc y estado antes de actuar.
- Si un mensaje viejo llega despues de un cambio de plan, se descarta por version/period key.
- Payment maneja reintentos de cobro; Subscription reacciona a PaymentSucceeded/Failed.
- Los avisos se publican a Notification; Subscription no envia email directamente.
- Eliminar QuartzDB y migrar solo el estado de negocio, no los registros internos de Quartz.

Paso 7 - Eventos

Consume	Publica
PaymentCompletedV1	SubscriptionPaymentRequestedV1
PaymentFailedV1	TenantProvisioningRequestedV1
TenantCreatedIntegrationEvent	SubscriptionActivatedV1
TenantStatusChangedIntegrationEvent	SubscriptionPastDueV1 / SuspendedV1
PlanChangePaymentCompletedV1	SubscriptionPlanChangedV1
	TenantEntitlementsChangedV1
	SubscriptionRenewalDueV1 / CancelledV1

TenantCreatedIntegrationEvent tambien llega para TaxVision Platform. El consumer de Subscription debe revisar Kind y devolver inmediatamente cuando sea Platform.

Paso 8 - Entitlements y limite de usuarios

Subscription publica TenantEntitlementsChangedV1 con IncludedUsers y FeatureCodes. Auth mantiene una proyeccion local. Al crear una invitacion TenantEmployee o TenantAdmin, Auth valida seats activos + invitaciones pendientes. CustomerPortal normalmente no consume una licencia interna, salvo que la politica comercial diga lo contrario.

No haga una llamada HTTP Auth -> Subscription durante CreateInvitation. La proyeccion local mantiene disponibilidad y sigue el mismo patron que Auth ya usa para proyectar tenants.

Paso 9 - API

Endpoint	Acceso
GET /plans	Publico, cacheado; solo versiones publicadas.
POST /subscription-enrollments	Publico rate-limited; crea intento, no tenant.
GET /subscription-enrollments/{id}	Token de enrollment o PlatformAdmin.
GET /subscriptions/current	TenantAdmin; TenantId desde JWT.
POST /subscriptions/current/change-plan	TenantAdmin.
POST /subscriptions/current/cancel	TenantAdmin.
POST /plans y /plan-versions	PlatformAdmin.

Paso 10 - Persistencia y migracion

```
dotnet ef migrations add InitialSubscription `
--project src/Services/Subscription/TaxVision.Subscription.Infrastructure `
--startup-project src/Services/Subscription/TaxVision.Subscription.Api

dotnet ef migrations script --idempotent `
--project src/Services/Subscription/TaxVision.Subscription.Infrastructure `
--startup-project src/Services/Subscription/TaxVision.Subscription.Api
```

- Seed de planes mediante datos versionados o comandos PlatformAdmin, nunca mediante IDs aleatorios por ambiente.
- Indice unique Subscriptions.TenantId garantiza una suscripcion vigente por tenant en el modelo inicial.
- PaymentId, ProvisioningKey y external event id tienen restricciones unique para idempotencia.
- Los campos Status y BillingPeriod se almacenan como string.
- No agregar foreign keys a TenantDB, AuthDB o PaymentDB.

12. Migracion legacy de Subscription

Legacy	Destino / decision
Service	Plan + primera PlanVersion.
Module + ServiceModule	PlanFeatures por PlanVersion.
CustomPlan	Subscription tenant-owned con PricingSnapshot.
CustomModule	Entitlements/add-ons dentro del snapshot o SubscriptionItems si se facturan por separado.
PendingPlanChange	PendingSubscriptionChange tipado; no JSON libre salvo metadata no critica.
Transaction*	Payment/Billing, no Subscription.
PricingHistory/PricingItem	PlanVersion immutable; elimina historial mutable.
RenewalNotificationLog	Notification/Audit.
Quartz jobs y QuartzDB	Wolverine scheduled messages; no migrar tablas tecnicas.

12.1 Estrategia de datos

- Crear Plan y PlanVersion a partir de Service, conservando moneda y periodo.
- Mapear CompanyId legacy a TenantId mediante la tabla de equivalencia del cutover.
- Crear una Subscription por CustomPlan valido y congelar el precio acordado.
- Convertir IsCaceled a estado Cancelled o CancelAtPeriodEnd segun fechas.
- Migrar pendientes solo si tienen pago/referencia verificable; el resto se reconcilia manualmente.
- No disparar renovaciones durante el ETL; programarlas despues de validar periodos.
- Comparar conteos, total recurring revenue por moneda y fechas de renovacion antes del cutover.

12.2 Pruebas obligatorias de Subscription

Nivel	Casos
Unitarias	Transiciones, trial, cancel-at-period-end, proration, version inmutable y Platform tenant rechazado.
Integracion	Unique TenantId/PaymentId, outbox, scheduled messages y concurrencia de PaymentCompleted.
Consumer	Duplicados, eventos fuera de orden, TenantCreated Platform ignorado y retries.
E2E	Plan -> enrollment -> payment -> tenant -> invitation admin -> subscription activa.
Reconciliacion	MRR/ARR, subscriptions por estado, renewal dates y entitlements.

13. Integracion completa entre servicios

```

Public Web
-> SubscriptionEnrollment
  -> Payment
    -> Tenant Service (Kind.Customer)
      -> Auth Invitation(TenantAdmin)
        -> User accepts and logs in

Customer CRM
-> Customer Service
  -> Auth Invitation(CustomerPortal, CustomerId)
    -> Customer portal JWT

Subscription
-> TenantEntitlementsChangedV1
  -> Auth local entitlement projection
    -> seat validation during invitations
  
```

Dato	Fuente de verdad	Proyecciones permitidas
Tenant identity/status/timezone	Tenant Service	Auth, Subscription y otros consumidores.
User/password/actor/customer link	Auth	JWT firmado; no copiar password/hash.
Customer master/fiscal	Customer	Read models minimos, sin PII fiscal por defecto.
Plan/subscription/entitlements	Subscription	Auth y Gateway si necesitan limites.
Payment transaction	Payment	Subscription conserva PaymentId y resultado.
Invoice/accounting	Billing	Subscription no duplica ledger.

13.1 Reglas de seguridad cross-service

- Nunca confiar en TenantId o CustomerId enviados libremente cuando existe un claim firmado.
- PlatformAdmin no cambia su tenant_id para operar como otro tenant; usa endpoints control-plane explicitos.
- Impersonation, si se agrega, usa token temporal, motivo, aprobacion y audit log separado.
- CustomerPortal solo accede al customer_id de su JWT dentro del tenant_id firmado.
- Eventos publicos no transportan secrets, passwords, invitation raw tokens o PII fiscal completa.
- Cada consumidor guarda solo los campos que necesita y trata el evento como contrato versionado.

14. Orden de implementacion recomendado

Iteracion	Entregable	Criterio de salida
0	Cerrar Auth invitations y tenant Platform	Migraciones, bootstrap, actor claims y README verificados.
1	Customer scaffolding + aggregate central	Factory, VOs, relaciones y unit tests.
2	Customer persistence + API minima	Create/Get/Search/Archive con tenant filter.
3	Fiscal profiles y cifrado	Acceso separado, blind index y audit.
4	Customer portal con Auth	E2E invitation -> login -> customer_id.
5	Customer events/read models	Outbox/inbox, consumers idempotentes.
6	Subscription plans + enrollment	PlanVersion inmutable y flujo pre-tenant.
7	Payment/Tenant provisioning	E2E Subscription -> Tenant -> Auth.
8	Lifecycle, proration y scheduled messages	Renewal, past due, cancel y plan change.
9	ETL y cutover	Reconciliacion aprobada y rollback ensayado.

14.1 Checklist de PR

- El diff no modifica archivos ajenos al slice.
- No hay secrets, tokens o PII en fixtures, logs o eventos.
- Las migrations Down no pierden datos evitables.
- El modelo snapshot coincide con configuraciones EF.
- Build y migration script idempotente terminan sin error.
- Los endpoints sensibles tienen Authorize/AllowAnonymous explicito y rate limiting.
- La documentacion y ejemplos JSON coinciden con nombres reales.
- Se incluyen riesgos, rollout, metricas y rollback.

15. Decisiones que no deben revertirse

No hacer	Motivo
Recrear ContactInfo como tabla cajon de sastre	Volveria a mezclar contacto, login y compliance.
Guardar PortalUserId en Customer	Invierte la dependencia; Auth ya conoce CustomerId.
Mantener Spouse y Dependent como tablas separadas	CustomerRelation cubre ambos con proposito tipado.
Mezclar RelationshipKind con ContactType	Parentesco y uso operacional son dimensiones diferentes.
Poner CustomerNotes en el aggregate	Acopla actividad operacional a identidad maestra.
Guardar fiscal profile como columnas en Customers	Aumenta exposicion, permisos y responsabilidad del root.
Crear tenant antes de enrollment/pago	Deja tenants huérfanos y complica compensaciones.
Llamar Auth -> Subscription por HTTP para seats	Reduce disponibilidad; usar proyeccion de entitlements.
Copiar transactions a Subscription	Payment/Billing son las fuentes monetarias.
Dar password comun a admins	Un compromiso escala a todos los tenants.

16. Resumen ejecutivo de implementacion

Customer queda como aggregate central del expediente maestro, con seis tablas principales y una entidad CustomerRelation que representa conyuges, dependientes, contactos y socios sin mezclar parentesco con funcion. La informacion fiscal se aísla en CustomerFiscalProfile y CustomerRelationFiscalProfile. Auth conserva el vinculo de portal mediante CustomerId; CustomerNotes y PortalUserId no vuelven al bounded context.

Subscription queda separado de Payment, Billing y Tenant. Un Enrollment pre-tenant permite que la suscripcion o trial se confirme antes de crear la oficina. El tenant comercial nace por evento, Auth crea su invitacion TenantAdmin y Subscription se activa de forma idempotente. TaxVision Platform queda expresamente excluido.

La implementacion debe avanzar por slices verticales pequenos, cada uno con dominio, persistencia, API, eventos, pruebas y documentacion. Esa disciplina mantiene la uniformidad de TaxVision mientras crece hacia nuevos microservicios.